

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: HỆ THỐNG NHÚNG VÀ THIẾT KẾ GIAO TIẾP NHÚNG
ĐỀ TÀI:
THIẾT KẾ VÀ PHÁT TRIỂN SẢN PHẨM
BÀN PHÍM TIÊU CHUẨN ANSI 61 PHÍM

Giảng viên hướng dẫn: TS. Võ Lê Cường

Nhóm sinh viên: Nhóm 8

Họ và tên	Mã số sinh viên
Nguyễn Long Vũ	20224210
Cao Nguyễn Duy Đức	20223692
Trần Khánh Duy	20223767

Hà Nội, 06 /2026

Bảng 1: Phân công công việc chi tiết trong nhóm

Tuần	Nội dung công việc chi tiết	Người phụ trách	Hoàn thành
1	Khảo sát các dòng bàn phím cơ 60%/61%, nghiên cứu cấu trúc Key Matrix, USB HID và lựa chọn hướng thực hiện đề tài.	Cao Nguyễn Duy Đức	100%
2	Tìm hiểu STM32F103C8T6, USB HID Class, nghiên cứu tài liệu, Github, Datasheet và các firmware mã nguồn mở (QMK, TMK).	Cao Nguyễn Duy Đức	100%
3	Xây dựng yêu cầu thiết kế, lựa chọn số lượng phím, bố trí layout 61 phím và xây dựng sơ đồ khối tổng quát của hệ thống.	Cao Nguyễn Duy Đức	100%
4	Thiết kế sơ đồ nguyên lý (Schematic): Matrix 5×14, Switch, Diode chống Ghosting, kết nối STM32 Blue Pill.	Trần Khánh Duy	100%
5	Hoàn thiện sơ đồ nguyên lý, kiểm tra kết nối GPIO, nguồn cấp, USB D+/D- và rà soát lỗi thiết kế.	Trần Khánh Duy	100%
6	Lắp ráp phần cứng thử nghiệm, kiểm tra từng hàng/cột của ma trận phím và xác nhận hoạt động của các switch.	Cao Nguyễn Duy Đức, Trần Khánh Duy	100%
7	Phát triển thuật toán quét ma trận (Matrix Scan), kiểm tra đọc trạng thái từng phím trên phần cứng thực tế.	Nguyễn Long Vũ	100%
8	Thiết kế và tối ưu thuật toán Debounce bằng bộ đếm (Counter-based Debounce), loại bỏ hiện tượng rung tiếp điểm.	Nguyễn Long Vũ	100%
9	Kiểm thử Matrix Scan + Debounce, đánh giá độ ổn định khi nhấn nhanh, nhấn nhiều phím đồng thời và hiệu chỉnh thông số.	Nguyễn Long Vũ	100%

10	Xây dựng Key Mapping, hỗ trợ Layer 0/Layer Fn, xử lý các phím Modifier và tổ hợp phím chức năng.	Nguyễn Long Vũ	100%
11	Thiết kế HID Report theo chuẩn USB HID Keyboard, xây dựng module USB HID Device và truyền dữ liệu lên máy tính.	Nguyễn Long Vũ	100%
12	Tối ưu USB Endpoint, cấu hình Polling Rate 100 Hz, kiểm tra khả năng Plug & Play trên Windows và Linux.	Nguyễn Long Vũ	100%
13	Hoàn thiện cơ chế 6KRO, xử lý Overflow khi vượt quá 6 phím thường, kiểm thử các tổ hợp phím Modifier.	Nguyễn Long Vũ	100%
14	Kiểm thử toàn bộ hệ thống: Ghosting, Debounce, Layer Fn, USB HID, đánh giá độ ổn định và sửa lỗi phát sinh.	Nguyễn Long Vũ Trần Khánh Duy Cao Nguyễn Duy Đức	100%
15	Hoàn thiện firmware, tối ưu mã nguồn, rà soát tài liệu kỹ thuật, chuẩn bị hình ảnh, lưu đồ và kết quả thực nghiệm, hoàn thiện xây dựng thiết kế vật lý của sản phẩm	Cao Nguyễn Duy Đức Trần Khánh Duy Nguyễn Long Vũ	100%
16	Tổng hợp báo cáo, hoàn thiện slide thuyết trình, phân công nội dung trình bày và diễn tập bảo vệ đồ án.	Cao Nguyễn Duy Đức Trần Khánh Duy	100%

MỤC LỤC

1. Lời mở đầu.....	6
2. Survey sản phẩm cho đề tài thiết kế bàn phím 61 phím	8
2.1. Phân tích yêu cầu của bàn phím Gaming	8
2.1.1. Thông số kỹ thuật chung (General Specs)	8
2.1.2. Kết nối (Connectivity)	9
2.1.3. Yêu cầu hệ thống (System Requirements)	9
2.2. Các vấn đề khi phát triển bàn phím Gaming	9
2.2.1. Thách thức về Kỹ thuật Điện tử và Tín hiệu	10
2.2.2. Khó khăn trong Phát triển Firmware	11
2.2.3. Vấn đề Chế tạo và Vật liệu.....	11
2.2.4. Kiểm soát Chất lượng (Quality Control).....	12
2.3. Lý do chi tiết lựa chọn phân khúc bàn phím văn phòng	14
2.4. Phân tích và đưa ra lựa chọn.....	17
2.5. Yêu cầu chức năng và phi chức năng.....	17
2.5.1. Yêu cầu chức năng.....	17
2.5.2. Yêu cầu phi chức năng.....	18
3. Xác định yêu cầu đề tài: Giao tiếp giữa hai thiết bị	18
3.1. Thiết bị gửi (Peripheral): Bàn phím cơ STM32.....	19
3.2. Thiết bị nhận (Host): Máy tính điều khiển	19
3.3. Các ràng buộc và tiêu chuẩn kỹ thuật chi tiết.....	19
4. Giới thiệu chung về phần cứng	20
4.1. Đặc tính kỹ thuật cốt lõi (Core & Performance).....	20
4.2. Các bộ ngoại vi phục vụ giao tiếp bàn phím	20
5. Phân tích chi tiết Ma trận phím (Key Matrix) và Layout.....	22
5.1. Thiết kế Layout 60% ANSI	22
5.2. Kiến trúc ma trận phím 5x14	22
5.3. Cơ chế quét và giải quyết hiện tượng Ghosting.....	24
5.4. Quy trình quét ma trận thực tế (theo Firmware)	24
6. Thiết kế phần mềm (Firmware Design).....	25
6.1. Thuật toán quét và khử nhiễu (Matrix & Debounce)	26
6.1.1. Cơ chế quét ma trận Active LOW	26
6.1.2. Thuật toán Integrator Debouncing.....	26
6.2. Xử lý logic bàn phím (Keyboard Processor)	28
6.2.1. Quản lý Layer và Phím chức năng (Fn).....	28
6.2.2. Cơ chế 6-Key Rollover (6KRO) và Modifiers	30
6.3. Giao tiếp USB HID và Tối ưu hóa băng thông.....	32
7. Đánh giá kết quả dự án	35
7.1. Kết quả hoàn thiện của sản phẩm.....	36

DANH MỤC BẢNG BIỂU

Bảng 1: Phân công công việc chi tiết trong nhóm	2
Bảng 2: Các tính năng mà nhóm học hỏi từ bàn phím Aula F65.....	14
Bảng 3: Đánh giá kết quả thực hiện các công việc trong dự án.....	35
Bảng 4: Bảng thống kê phần trăm sử dụng AI trong các chương của báo cáo.....	36

DANH MỤC HÌNH ẢNH

Hình 1: Bàn phím gaming Logitech G Pro	10
Hình 2: Mô tả hiện tượng Debounce ở Switch	10
Hình 3: Bàn phím công thái học (ergonomic) với vỏ bằng gỗ óc chó	11
Hình 4: Bàn phím gaming Razer BlackWidow Chroma V2	12
Hình 5: Bàn phím Aula F65 trên thị trường	13
Hình 6: Vi điều khiển STM32F103C8	21
Hình 7: Schematic của Keyboard 14 x 5	23
Hình 8: Mặt trước layout PCB Keyboard 14 x 5	23
Hình 9: Mặt sau layout PCB Keyboard 14 x 5	23
Hình 10: Layout PCB 3D của sản phẩm	24
Hình 11: Lưu đồ thuật toán của quy trình quét ma trận	25
Hình 11: Sản phẩm khi đã hoàn thiện	36

1. Lời mở đầu

Trong kỷ nguyên số hóa hiện nay, bàn phím không còn đơn thuần là một thiết bị ngoại vi nhập liệu cơ bản, mà đã trở thành một giao diện tương tác người - máy (Human-

Computer Interface) quan trọng, ảnh hưởng trực tiếp đến hiệu suất làm việc và trải nghiệm người dùng. Đối với một kỹ sư hệ thống nhúng, việc thiết kế một bàn phím cơ từ con số không (from scratch) không chỉ là một bài tập thực hành, mà còn là một thử thách tổng hợp đòi hỏi sự kết hợp nhuần nhuyễn giữa tư duy thiết kế phần cứng tối ưu và kỹ thuật lập trình firmware hiệu suất cao.

Dự án này tập trung vào việc hiện thực hóa một bàn phím cơ chuẩn layout 60% tối giản – một xu hướng đang rất được ưa chuộng nhờ sự gọn gàng nhưng vẫn đảm bảo đầy đủ các tính năng cần thiết thông qua hệ thống phân tầng (layers). Để giải quyết bài toán này, vi điều khiển STM32F103C8T6 (thường được biết đến với tên gọi Blue Pill) dựa trên kiến trúc ARM Cortex-M3 đã được lựa chọn làm "trái tim" của hệ thống. Với tốc độ xử lý 72 MHz và bộ ngoại vi hỗ trợ USB Full-speed tích hợp, STM32 cung cấp sức mạnh vượt trội so với các dòng MCU 8-bit truyền thống, cho phép thực hiện các thuật toán phức tạp với độ trễ cực thấp.

Báo cáo này sẽ trình bày một quy trình thiết kế và chế tạo toàn diện, bao gồm:

- Thiết kế phần cứng: Từ việc xây dựng sơ đồ nguyên lý cho ma trận phím 5 x 14, cách bố trí Diode chống hiện tượng ghosting, đến việc thiết kế mạch in (PCB) đảm bảo tính toàn vẹn tín hiệu cho đường truyền USB.
- Xử lý tín hiệu nhúng: Áp dụng thuật toán khử nhiễu tiếp điểm (Debouncing) theo phương pháp đếm tích phân (Integrator Counter) để loại bỏ hoàn toàn các xung nhiễu vật lý của switch cơ học.
- Giao thức truyền thông: Hiện thực hóa chuẩn USB HID (Human Interface Device) để thiết bị có khả năng tương tác trực tiếp với mọi hệ điều hành máy tính mà không cần cài đặt trình điều khiển (driver) riêng biệt.
- Tối ưu hóa logic: Xây dựng hệ thống quản lý phím đa tầng (Multi-layer keymap) và hỗ trợ tính năng 6KRO (6-Key Rollover), đảm bảo bàn phím có thể phản hồi chính xác ngay cả trong các tác vụ đòi hỏi tốc độ cao như chơi game hay lập trình.

Thông qua đề tài này, người thực hiện không chỉ nắm vững quy trình phát triển sản phẩm nhúng thực tế mà còn hiểu sâu sắc về cách thức tối ưu hóa sự tương tác giữa

phần cứng vật lý và phần mềm điều khiển để tạo ra một thiết bị ngoại vi ổn định và chuyên nghiệp.

2. Survey sản phẩm cho đề tài thiết kế bàn phím 61 phím

2.1. Phân tích yêu cầu của bàn phím Gaming

Sau khi tham khảo các datasheet của các hãng sản xuất bàn phím gaming lớn hiện nay như **Razer, Corsair, Asus, Keychron, Gigabyte, Kingston hay Aula**, nhóm nhận thấy rằng bàn phím gaming hiện nay tập trung mạnh vào hiệu năng thời gian thực nhằm đáp ứng các game FPS, MOBA và Esports.

Các yêu cầu kỹ thuật chính bao gồm:

2.1.1. Thông số kỹ thuật chung (General Specs)

- **Loại Switch (Công tắc phím):** Hầu hết các bàn phím gaming hiện nay sử dụng switch cơ học (Mechanical) như **Cherry MX (Đỏ, Nâu, Xanh)**, Kailh Red hoặc Razer Mechanical. Ngoài ra, còn có các loại switch hiện đại hơn như Optical-Mechanical (Quang-Cơ) giúp tăng tốc độ phản hồi hoặc Magnetic Switch (Switch từ tính/Hall Effect) cho phép tùy chỉnh điểm nhận lệnh.
- **Tính năng Anti-Ghosting và N-Key Rollover (NKRO):** Đây là tiêu chuẩn bắt buộc, cho phép bàn phím nhận diện chính xác nhiều phím nhấn cùng lúc (thường là toàn bộ phím hoặc tối thiểu 10-19 phím đồng thời) để tránh bị mất lệnh khi chơi game ở tốc độ cao.
- **Tần suất gửi tín hiệu (Polling Rate):** Phổ biến nhất là 1000Hz (Ultrapolling), nghĩa là bàn phím gửi tín hiệu đến máy tính 1000 lần mỗi giây (độ trễ 1ms). Một số dòng cao cấp có thể hỗ trợ tốc độ cao hơn nữa.
 - Hệ thống chiếu sáng: Đèn nền RGB với 16.8 triệu màu là tính năng gần như mặc định, cho phép tùy chỉnh từng phím hoặc theo vùng (zones) và đồng bộ hóa với các thiết bị khác thông qua phần mềm.
 - Khả năng lập trình và Macro: Hỗ trợ lập trình phím và ghi lại Macro (chuỗi lệnh) trực tiếp (on-the-fly) hoặc thông qua phần mềm để tối ưu hóa thao tác trong game.
 - Thiết kế và Độ bền:
 - Tuổi thọ phím thường từ 50 triệu đến 80 triệu lần nhấn.
 - Sử dụng vật liệu bền bỉ như khung thép hoặc nhựa chất lượng cao, đi kèm keycap PBT double-shot để chống mài mòn.

- Hỗ trợ quản lý dây cáp và thường đi kèm kê tay (wrist rest) có thể tháo rời để tạo sự thoải mái.

2.1.2. Kết nối (Connectivity)

- Dây (Wired): Sử dụng cổng USB 2.0 hoặc 3.0 (Type-A hoặc Type-C). Nhiều bàn phím có thêm cổng USB pass-through để cắm trực tiếp chuột hoặc tai nghe vào bàn phím.
- Không dây (Wireless): Các mẫu hiện đại thường hỗ trợ Tri-mode (3 chế độ): Bluetooth (có thể kết nối cùng lúc 3 thiết bị), Wireless 2.4GHz (độ trễ thấp cho gaming) và kết nối có dây qua Type-C.

2.1.3. Yêu cầu hệ thống (System Requirements)

- Hệ điều hành:
 - Windows: Tối thiểu Windows 7, nhưng phổ biến nhất hiện nay yêu cầu Windows 10 hoặc Windows 11 để hoạt động đầy đủ tính năng phần mềm.
 - Mac: Mac OS X 10.9 hoặc cao hơn.
- Phần cứng: Cần ít nhất một cổng USB trống. Một số bàn phím yêu cầu 2 đầu cắm USB nếu sử dụng tính năng USB pass-through.
- Phần mềm quản lý: Cần có kết nối Internet để tải về và cài đặt các phần mềm điều khiển như Razer Synapse, Corsair iCUE, ASUS Armoury Crate hoặc HyperX NGenuity.
- Bộ nhớ lưu trữ: Cần khoảng 200 MB dung lượng ổ cứng trống để cài đặt driver và phần mềm điều khiển.

2.2. Các vấn đề khi phát triển bàn phím Gaming

Việc phát triển, thiết kế và sản xuất một bàn phím gaming là một quá trình phức tạp, đòi hỏi sự kết hợp giữa kỹ thuật điện tử, lập trình phần mềm (firmware) và chế tạo cơ khí.

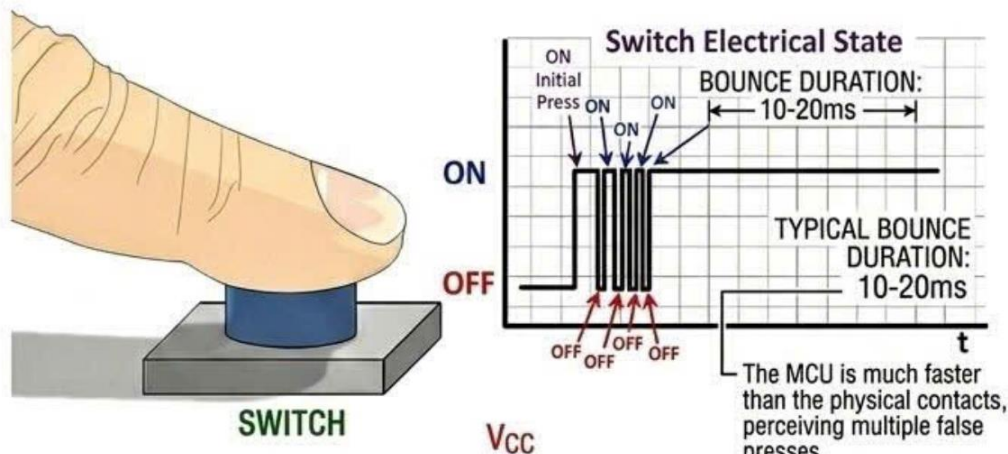


Hình 1: Bàn phím gaming Logitech G Pro

2.2.1. Thách thức về Kỹ thuật Điện tử và Tín hiệu

- **Hiện tượng dội phím (Contact Bounce):** Khi sử dụng switch cơ học, tiếp điểm kim loại sẽ có hiện tượng rung động khi nhấn, gây ra nhiều tín hiệu giả. Vậy nên sản phẩm cần thiết kế mạch khử nhiễu (**debouncing circuits**) hoặc thuật toán firmware để đảm bảo mỗi lần nhấn chỉ ghi nhận một lệnh.

SWITCH DE-BOUNCING



Hình 2: Mô tả hiện tượng Debounce ở Switch

- **Hiện tượng bóng ma (Ghosting):** Trong thiết kế ma trận phím (matrix scanning), nếu không có các diode tại mỗi vị trí phím, hệ thống có thể ghi nhận nhầm các phím ảo khi nhấn nhiều phím cùng lúc. Điều này đặc biệt quan trọng với bàn phím gaming cần tính năng N-Key Rollover (NKRO).
- **Quản lý năng lượng và kết nối không dây:** Nếu phát triển bàn phím không dây, việc chọn pin, thiết kế mạch sạc và mạch chia tải (load sharing) để chuyển

đổi mượt mà giữa nguồn pin và USB là một thách thức lớn. Ngoài ra, việc thiết kế ăng-ten và đảm bảo độ trễ thấp cho chơi game cũng rất phức tạp.

2.2.2. Khó khăn trong Phát triển Firmware

- Giao tiếp với các ngăn xếp (stacks) Bluetooth: Việc tích hợp firmware tự viết với các bộ SDK của nhà sản xuất chip (như Nordic Semiconductor) rất khó khăn do tính phức tạp của các linker script và các ngắt ưu tiên cao (interrupts) mà lập trình viên không được phép can thiệp.
- Lựa chọn ngôn ngữ lập trình: Sử dụng các ngôn ngữ hiện đại như Rust có thể gặp nhiều lỗi về liên kết (linking) và bindings khi gọi các hàm C từ SDK cũ, đôi khi phải chuyển sang các ngôn ngữ nhỏ gọn hơn như Zig để tối ưu hóa.

2.2.3. Vấn đề Chế tạo và Vật liệu

- Gia công CNC và Đồ gá (Fixtures): Khi sản xuất vỏ bàn phím bằng vật liệu cao cấp như nhôm, gỗ hoặc Corian, việc thiết kế đồ gá để giữ phôi khi gia công cả hai mặt phím và cắt biên dạng là một thử thách cơ khí. Tốc độ cắt và bước tiến dao phải được tinh chỉnh để tránh làm hỏng bề mặt vật liệu.



Hình 3: Bàn phím công thái học (ergonomic) với vỏ bằng gỗ óc chó

- Đúc khuôn chi tiết nhỏ: Việc tạo ra các đế bàn phím bằng silicone hoặc polyurethane có thể gặp vấn đề về độ nhớt (viscosity), bọt khí bị kẹt bên trong, hoặc phản ứng hóa học không mong muốn giữa nhựa và khuôn.
- Sản xuất Keycap: Các đặc điểm có tỷ lệ chiều cao lớn (như chân stem của keycap) rất khó chế tạo bằng in 3D (dễ gãy) hoặc gia công CNC truyền thống;

chúng thường đòi hỏi đúc phun (injection molding) với khuôn kim loại phức tạp.

2.2.4. Kiểm soát Chất lượng (Quality Control)

- Tiêu chuẩn công nghiệp: Bàn phím cần đạt các tiêu chuẩn **về chống tĩnh điện (ESD), tương thích điện từ (EMI/EMC)** và các xếp hạng bảo vệ (IP) nếu muốn sử dụng trong các môi trường khắc nghiệt.

Nhận xét: Bàn phím gaming đòi hỏi kỹ thuật cao như polling rate 1000Hz và mạch NKRO phức tạp. Việc lập trình firmware cho hệ thống RGB và Macro cũng tiêu tốn nhiều nguồn lực phát triển. Thử thách cơ khí còn nằm ở việc gia công CNC và đúc keycap chi tiết nhỏ.



Hình 4: Bàn phím gaming Razer BlackWidow Chroma V2

Trong khi đó, bàn phím cơ dành cho văn phòng và công việc có các yêu cầu phù hợp hơn với phạm vi đồ án.

→ Chọn phát triển bàn phím cơ 61 phím dành cho văn phòng và công việc. Cụ thể nhóm đã chọn sản phẩm Aula F65.



Hình 5: Bàn phím Aula F65 trên thị trường

2.3. Lý do chi tiết lựa chọn phân khúc bàn phím văn phòng

Bảng 2: Các tính năng mà nhóm học hỏi từ bàn phím Aula F65

Tiêu chí	Bàn phím văn phòng Aula F65	Lý do chọn phân khúc văn phòng
Tần suất quét (Scan Rate)	100 Hz (đủ cho tác vụ văn phòng thông thường)	Giảm áp lực lên vi xử lý và đơn giản hóa thiết kế phần cứng
Khả năng nhận diện phím (Rollover)	6-key (6KR)	Thiết kế mạch điện (matrix) đơn giản hơn rất nhiều, không cần quá nhiều linh kiện phụ trợ
Giao tiếp (Interface)	Đơn giản: chủ yếu chạy chuẩn USB HID tiêu chuẩn, phím chức năng cơ bản,.	Né được việc phải xử lý các ngăn xếp (stacks) Bluetooth/USB phức tạp và lỗi linker script
Vật liệu & Gia công	Sử dụng nhựa tiêu chuẩn (ABS, Polycarbonate), thiết kế tối ưu cho đúc khuôn hàng loạt,.	Quy trình sản xuất ổn định, chi phí vật liệu và gia công rẻ hơn khi làm số lượng lớn
Thiết kế & Công năng	Tập trung vào Công thái học (Ergonomics), phím gõ êm (quiet keys), layout chia vùng,..	Người dùng văn phòng ưu tiên sự thoải mái và sức khỏe, đây là giá trị thực tế dễ tiếp cận hơn là chạy đua tốc độ
Rủi ro sản xuất	Thấp: Sản phẩm có tính ứng dụng cao, dễ dàng tận dụng linh kiện có sẵn trên thị trường.	Đảm bảo tính khả thi cho dự án sinh viên, dễ dàng kiểm soát chất lượng đầu ra

Bảng tổng kết các Github liên quan đến đề tài

STT	Tên dự án	Mô tả mục tiêu	Chi tiết chức năng	Công nghệ / Phương pháp sử dụng	Kết quả đạt được	Ưu điểm	Nhược điểm	Link GitHub
1	keyboar d-pcb- guide	Hướng dẫn mang tính giáo dục về quy trình thiết kế PCB bàn phím cơ từ đầu bằng KiCad.	Thiết kế sơ đồ nguyên lý (MCU, thạch anh, tụ điện, cổng USB); xây dựng ma trận phím 2x2 cơ bản; kỹ thuật vẽ mạch in (routing) và xuất file Gerber để sản xuất.	Phần mềm KiCad; vi điều khiển ATmega32 U4; linh kiện dán (SMD); kỹ thuật tạo mặt phẳng đất (Ground Plane).	Bộ tài liệu hướng dẫn hoàn chỉnh kèm file thiết kế mẫu cho mạch 2x2.	Tài liệu cực kỳ chi tiết từng bước, giải thích rõ các thuật ngữ kỹ thuật, phù hợp nhất cho người mới bắt đầu.	Công nghệ hơi cũ (dùng USB Mini/Micro-B thay vì USB-C); thiết kế 2x2 quá đơn giản để ứng dụng làm sản phẩm thực tế.	ruiqima/keyboard-pcb-guide
2	keyboar d-labs	Bộ sưu tập khổng lồ các thiết kế bàn phím mã nguồn mở, hiện đại và linh hoạt.	Hỗ trợ nhiều layout (Split X-1, Ortholinear X-2, Pico42); tính năng cao cấp: đèn LED RGB từng phím, HotSwap, màn hình OLED, hỗ trợ Bluetooth (BLE) và sạc pin LiPo.	Đa dạng MCU (STM32/ARM, RP2040, RISC-V); Firmware: QMK, Rust (keyberon), Nickel (fak); sử dụng script để tự động tạo file CAD từ PCB.	Một hệ sinh thái gồm nhiều bản thiết kế chuyên nghiệp có thể sản xuất và sử dụng ngay.	Áp dụng công nghệ tiên tiến (RISC-V, Rust); đa dạng lựa chọn linh kiện; có kịch bản tự động hóa thiết kế vỏ máy.	Rất phức tạp; yêu cầu kiến thức sâu về nhiều ngôn ngữ lập trình và quy trình cài đặt môi trường (toolchain).	rngoulter/keyboard-labs

3	PCB-Design (Orions Hands)	Thiết kế nguyên mẫu bàn phím cơ tùy chỉnh tích hợp màn hình và nút xoay.	Hỗ trợ NKRO (ấn nhiều phím cùng lúc); nút xoay (rotary encoder) điều khiển đa phương tiện qua USB; màn hình I2C SSD1309; xử lý đa nhân (multicore) để tăng tốc độ hiển thị.	Ngôn ngữ Rust (65%) và C; vi điều khiển RP Pico (RP2040); thư viện embedded-graphics để xử lý màn hình.	Bản mẫu bàn phím thực tế (Orions Hands V0.1) hoạt động với firmware viết hoàn toàn bằng Rust.	Firmware hiệu suất cao nhờ tận dụng tối đa sức mạnh đa nhân của RP2040; tích hợp hiển thị đồ họa sáng tạo.	Có lỗi phần cứng gây hiện tượng "ghosting" sau thời gian dài sử dụng; việc độ lại cổng USB-C khá khó khăn.	Allorx/PCB-Design
4	Emu-Board	Xây dựng hệ thống bàn phím nhúng tùy chỉnh "gần như hoàn toàn từ đầu".	Bộ cục 77 phím; vỏ máy thiết kế riêng để cắt laser; hoạt động như một thiết bị USB HID chuẩn.	Firmware KMK (dựa trên Python/CircuitPython); ngôn ngữ Ruby (để sinh file CAD cắt laser vỏ máy); KiCad; RP Pico.	Sản phẩm hoàn thiện cả về phần cứng, phần mềm và vỏ máy acrylic.	Tập trung mạnh vào khả năng tùy biến ngoại hình và firmware linh hoạt, dễ sửa đổi bằng Python.	Tài liệu hướng dẫn còn ít; phụ thuộc vào nhiều ngôn ngữ (Ruby, Python) gây khó khăn khi thiết lập môi trường đồng bộ.	Emmanuel-Roy/Emu-Board

5	keycool84keyboard	Tự xây dựng bàn phím cơ dựa trên bố cục thương mại để hiểu sâu nguyên lý hoạt động.	Bố cục 84 phím (compact); ma trận phím 6x14 có diode chống dòng ngược; hỗ trợ NKRO; vỏ máy kết hợp plate thép và linh kiện in 3D.	Ngôn ngữ C++ (Arduino); module Arduino Pro Micro; KiCad; Fusion 360; phần mềm Autorouter (Freerouter)	Sản phẩm thực tế với PCB trắng tự thiết kế và tấm plate thép không gỉ được gia công riêng.	Giải thích cực kỳ chi tiết các lý do kỹ thuật (tại sao dùng diode, cách quét ma trận); quy trình thực hiện rất thực tế cho sinh viên.	Vỏ máy chưa bao phủ toàn bộ linh kiện (dùng ốc vít làm chân đế); chưa tích hợp MCU trực tiếp lên mạch in mà dùng module rời.	overlordpro-sys/keycool84keyboards
---	-------------------	---	---	---	--	---	--	--

2.4. Phân tích và đưa ra lựa chọn

Lựa chọn cuối cùng: **Dự án STT 5 - keycool84keyboard (của overlordpro-sys).**

Lý do lựa chọn:

- Tính toàn diện và giáo dục:** Dự án này không chỉ cung cấp mã nguồn mà còn giải thích chi tiết "hành trình" thiết kế, từ việc chọn bố cục (layout), cách đi dây ma trận dạng lưới để tiết kiệm chân pin, đến việc sử dụng diode để tránh lỗi phím ma. Điều này rất phù hợp với yêu cầu của một Project môn học.
- Công nghệ phù hợp với trình độ:** Sử dụng Arduino Pro Micro và ngôn ngữ C++ là những nền tảng cực kỳ phổ biến trong môi trường học tập, giúp giảm thiểu rủi ro gặp lỗi phần mềm khó xử lý.
- Quy trình kỹ thuật chuẩn:** Dự án hướng dẫn đầy đủ từ khâu thiết kế sơ đồ, vẽ mạch PCB, đặt hàng sản xuất tại JLCPCB, đến khâu thiết kế cơ khí (Fusion 360) và in 3D.
- Tính khả thi cao:** Việc sử dụng các thư viện Keypad và Keyboard của Arduino giúp rút ngắn thời gian phát triển firmware, cho phép sinh viên tập trung nhiều hơn vào phần thiết kế phần cứng trong một học kỳ.

2.5. Yêu cầu chức năng và phi chức năng

2.5.1. Yêu cầu chức năng

- **Quét ma trận phím (Matrix Scanning):** MCU phải liên tục kiểm tra trạng thái của ma trận 5x14 để không bỏ lỡ bất kỳ thao tác nhấn phím nào từ người dùng.
- **Khử nhiễu phím (Debouncing):** Sử dụng thuật toán đếm tích phân (Integrator Counter) để loại bỏ các xung nhiễu vật lý phát sinh khi switch cơ học tiếp xúc.
- **Định danh mã phím (Keycode Mapping):** Chuyển đổi tọa độ vật lý (hàng, cột) sang mã HID chuẩn.
- **Quản lý đa tầng (Multi-layer):** Hỗ trợ phím Fn để kích hoạt Layer 1, cho phép các phím vật lý đảm nhận nhiều chức năng khác nhau (như F1-F12).
- **Cơ chế 6-Key Rollover (6KRO):** Có khả năng nhận diện và gửi báo cáo tối đa 6 mã phím thường được nhấn đồng thời cùng với các phím Modifier (Ctrl, Shift, Alt, Gui).
- **Giao tiếp chuẩn USB HID:** Thiết bị phải tự đóng gói dữ liệu vào gói tin 8-byte theo đúng giao thức để máy tính có thể giải mã.

2.5.2. Yêu cầu phi chức năng

- **Độ bền cơ học:** Vỏ phải đảm bảo không bị biến dạng khi gõ, giúp bảo vệ các mối hàn diode và switch khỏi bị bong tróc.
- **Tính cách điện:** Lớp đáy vỏ phải đảm bảo cách điện tuyệt đối với mặt dưới của PCB (nơi có các chân linh kiện tự hàn) để tránh ngắn mạch.
- **Góc nghiêng công thái học:** Thiết kế vỏ hoặc chân đế phải tạo độ dốc từ 5° đến 7° để giảm mỏi cổ tay cho người dùng văn phòng.
- **Lỗ thoát cáp:** Phải có khoang chứa đủ rộng (khoảng 15mm) để chứa module Blue Pill và lỗ khoét cổng USB phải khớp hoàn toàn để đảm bảo kết nối ổn định.
- **Loại bỏ hệ thống đèn nền (Non-Backlight):** Dự án lược bỏ hoàn toàn hệ thống LED RGB để giảm thiểu việc đi dây phức tạp trên PCB tự thiết kế và tiết kiệm tài nguyên chân GPIO cho STM32.

3. Xác định yêu cầu đề tài: Giao tiếp giữa hai thiết bị

Mục tiêu trọng tâm của dự án là hiện thực hóa quy trình truyền tin từ hành vi cơ học (nhấn phím) đến dữ liệu số mà máy tính có thể hiểu được. Quá trình này đòi hỏi sự phối hợp chặt chẽ giữa thiết bị ngoại vi và máy chủ thông qua giao thức tiêu chuẩn.

3.1. Thiết bị gửi (Peripheral): Bàn phím cơ STM32

Thiết bị gửi đóng vai trò là một hệ thống nhúng thời gian thực, thực hiện các nhiệm vụ:

- Quét ma trận (Matrix Scanning): MCU phải liên tục kiểm tra trạng thái của ma trận phím 14 x 5 với chu kỳ 1ms để đảm bảo không bỏ lỡ bất kỳ thao tác nào.
- Xử lý tín hiệu sơ cấp: Thực hiện thuật toán khử nhiễu (Debounce) để lọc bỏ các xung nhiễu vật lý phát sinh do sự đàn hồi của lò xo bên trong switch.
- Định danh mã phím (Keycode Mapping): Chuyển đổi vị trí vật lý (hàng, cột) sang mã HID chuẩn dựa trên bảng Keymap đã thiết lập (bao gồm cả việc quản lý các tầng phím Fn).
- Đóng gói dữ liệu (HID Report Construction): Tổng hợp mã phím vào một gói tin (Report) có kích thước 8-byte theo đúng cấu trúc của giao thức USB HID.

3.2. Thiết bị nhận (Host): Máy tính điều khiển

Máy tính đóng vai trò là Host, thực hiện quản lý kết nối thông qua Driver HID có sẵn trong hệ điều hành:

- Nhận diện thiết bị (Enumeration): Ngay khi kết nối, Host sẽ yêu cầu thiết bị cung cấp các "Descriptors" để xác định đây là một bàn phím (Keyboard Class).
- Xử lý dữ liệu: Giải mã các gói tin 8-byte nhận được để thực thi các lệnh tương ứng như hiển thị ký tự, điều khiển âm thanh hoặc thực hiện các tổ hợp phím hệ thống.

3.3. Các ràng buộc và tiêu chuẩn kỹ thuật chi tiết

Để đảm bảo thiết bị hoạt động chuyên nghiệp và ổn định, hệ thống phải đáp ứng các tiêu chuẩn sau:

- Chuẩn giao tiếp USB Full Speed (12 Mbps):
 - Sử dụng hai đường tín hiệu vi sai (Differential signaling) D+(PA12) và D-(PA11) trên STM32.
 - Tốc độ này đảm bảo băng thông dư thừa cho việc gửi báo cáo phím liên tục mà không gây trễ.
- Cơ chế 6KRO (6-Key Rollover):

- Bàn phím phải có khả năng thông báo tối đa 6 mã phím thường nhấn đồng thời trong một báo cáo USB.
- Cơ chế này đòi hỏi thuật toán xử lý phải có khả năng quản lý mảng dữ liệu phím nhấn và xử lý tràn (Overflow) nếu người dùng nhấn quá 6 phím cùng lúc.
- Tính năng Plug and Play (PnP):
 - Thiết bị phải tuân thủ nghiêm ngặt HID Usage Table 1.12.
 - Sử dụng các cấu hình chuẩn để hệ điều hành có thể nhận diện ngay lập tức mà không cần cài đặt phần mềm bên thứ ba, đảm bảo tính tương thích cao trên mọi nền tảng.
- Hiệu suất truyền tin:
 - Sử dụng cơ chế truyền tin dựa trên sự thay đổi (Change-based reporting): Chỉ gửi dữ liệu mới khi có sự thay đổi trạng thái phím để tối ưu hóa băng thông USB và giảm tải xử lý cho máy tính.

4. Giới thiệu chung về phần cứng

4.1. Đặc tính kỹ thuật cốt lõi (Core & Performance)

Dựa theo thông số từ datasheet, vi điều khiển này sở hữu những ưu điểm vượt trội cho hệ thống nhúng thời gian thực:

- Kiến trúc ARM 32-bit: Hoạt động với tần số tối đa 72 MHz, mang lại hiệu suất 1.25 DMIPS/MHz. Tốc độ này cho phép MCU thực hiện quét ma trận phím và xử lý thuật toán khử nhiễu (Debounce) gần như tức thời, giảm thiểu tối đa độ trễ đầu vào (input lag).
- Bộ nhớ: Được trang bị 64 KB Flash (đủ để chứa firmware phức tạp với nhiều layer và hiệu ứng LED) và 20 KB SRAM, giúp việc quản lý các mảng dữ liệu ma trận và buffer truyền thông USB trở nên mượt mà.
- Điện áp hoạt động: Từ 2.0V đến 3.6V, phù hợp hoàn hảo với điện áp 3.3V tiêu chuẩn của các hệ thống hiện đại và tiết kiệm năng lượng.

4.2. Các bộ ngoại vi phục vụ giao tiếp bàn phím



Hình 6: Vi điều khiển STM32F103C8

Việc lựa chọn STM32F103C8 thay vì các dòng 8-bit truyền thống là do sự hỗ trợ mạnh mẽ từ các khối ngoại vi tích hợp sẵn:

- USB 2.0 Full Speed Interface:
 - Đây là bộ phận quan trọng nhất của đề tài. MCU tích hợp sẵn bộ thu phát USB (transceiver) hỗ trợ tốc độ 12 Mbit/s.
 - Nó hỗ trợ tối đa 8 Endpoint, cho phép cấu hình đồng thời nhiều giao thức như Keyboard, Consumer Control (phím media) và System Control (phím nguồn) trên cùng một kết nối vật lý.
- Hệ thống GPIO linh hoạt:
 - MCU cung cấp tới 37 chân I/O tốc độ cao. Trong dự án, các chân này được cấu hình làm ma trận 5\14.
 - Khả năng chịu dòng (Current sink/source) tốt giúp tín hiệu quét ma trận luôn rõ ràng, ít nhiễu.
- Bộ quản lý ngắt (NVIC):
 - Với 43 kênh ngắt ngoài, hệ thống có thể phản hồi ngay lập tức với các sự kiện quan trọng, đảm bảo tính thời gian thực cho việc truyền nhận dữ liệu USB mà không làm gián đoạn việc quét phím.
- Timers:

- Các bộ định thời 16-bit được sử dụng để tạo ra nhịp quét 1ms chính xác tuyệt đối, tạo nên tần số Polling Rate 1000Hz tiêu chuẩn cho các bàn phím chuyên dụng cao cấp.

5. Phân tích chi tiết Ma trận phím (Key Matrix) và Layout

5.1. Thiết kế Layout 60% ANSI

Layout 60% là một thiết kế tối giản nhưng cực kỳ khoa học. Qua hình ảnh thiết kế, bàn phím của dự án tuân thủ tiêu chuẩn 61 phím (ANSI) với các đặc điểm:

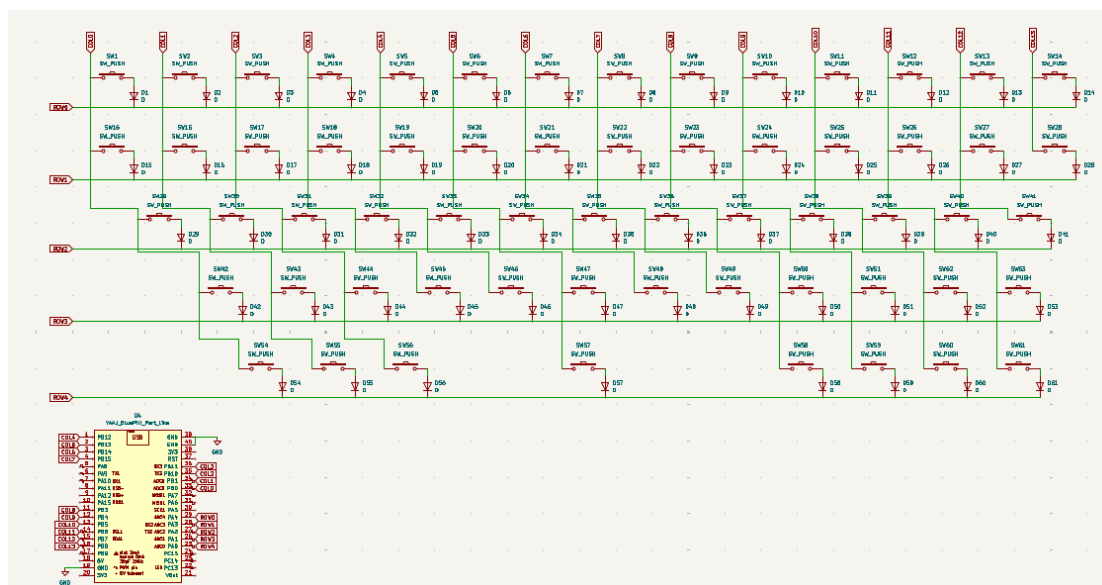
- Sự tối giản: Loại bỏ hoàn toàn cụm phím số (Numpad), hàng phím chức năng (F1-F12) và cụm phím điều hướng rời. Điều này giúp giảm kích thước bàn phím xuống chỉ còn khoảng 290mm x 100mm, tạo không gian rộng rãi cho di chuột.
- Tính thẩm mỹ và công thái học: Các phím có kích thước tiêu chuẩn (1U = 19.05mm), các phím đặc biệt như Shift, Enter, Space được tính toán chính xác đơn vị (độ dài 2.25U, 6.25U...) để đảm bảo sự cân đối khi lắp Keycap.
- Giải pháp bù đắp tính năng: Dù thiếu các phím vật lý, nhưng thông qua phím Fn (Function) được đặt tại góc dưới bên phải (theo file keymap.c), người dùng có thể kích hoạt Layer 1 để sử dụng đầy đủ các tính năng của một bàn phím Full-size.

5.2. Kiến trúc ma trận phím 5x14

Để quản lý 61 phím với số lượng chân GPIO hạn chế của STM32, hệ thống sử dụng cấu trúc ma trận giao điểm:

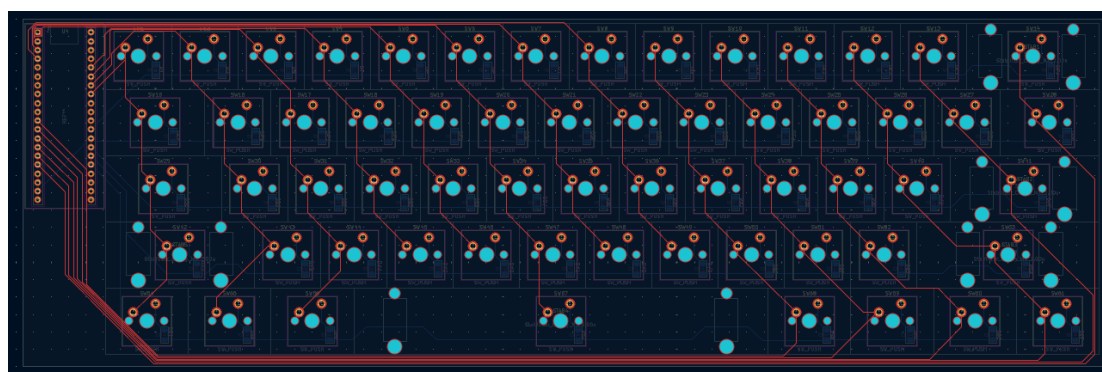
- Số lượng ô vật lý: Ma trận có 5 hàng x 14 cột = 70 điểm giao cắt. Thực tế chỉ sử dụng 61 điểm cho các phím, 9 điểm còn lại được khai báo là KC_NO trong firmware để MCU bỏ qua khi quét
- Cấu hình chân tín hiệu:
 - a) 5 Hàng (Rows): Kết nối với các chân từ PA0 đến PA4. Trong code, các chân này được cấu hình ở chế độ Output Push-Pull. Ở trạng thái nghỉ, chúng được giữ ở mức HIGH.
 - b) 14 Cột (Cols): Kết nối với các chân từ PB0 đến PB13. Các chân này được cấu hình là Input Pull-up. MCU sẽ đọc trạng thái tại đây để xác định phím nhấn.

Với cấu trúc như trên, nhóm đã thực hiện xây dựng schematic bằng phần mềm KiCad:



Hình 7: Schematic của Keyboard 14 x 5

- Mỗi chân switch được kết nối thêm một diode nhằm bảo vệ switch và mạch. Khi triển khai layout PCB, footprint của switch được chọn là switch cherry và diode SMD:



Hình 8: Mặt trước layout PCB Keyboard 14 x 5



Hình 9: Mặt sau layout PCB Keyboard 14 x 5



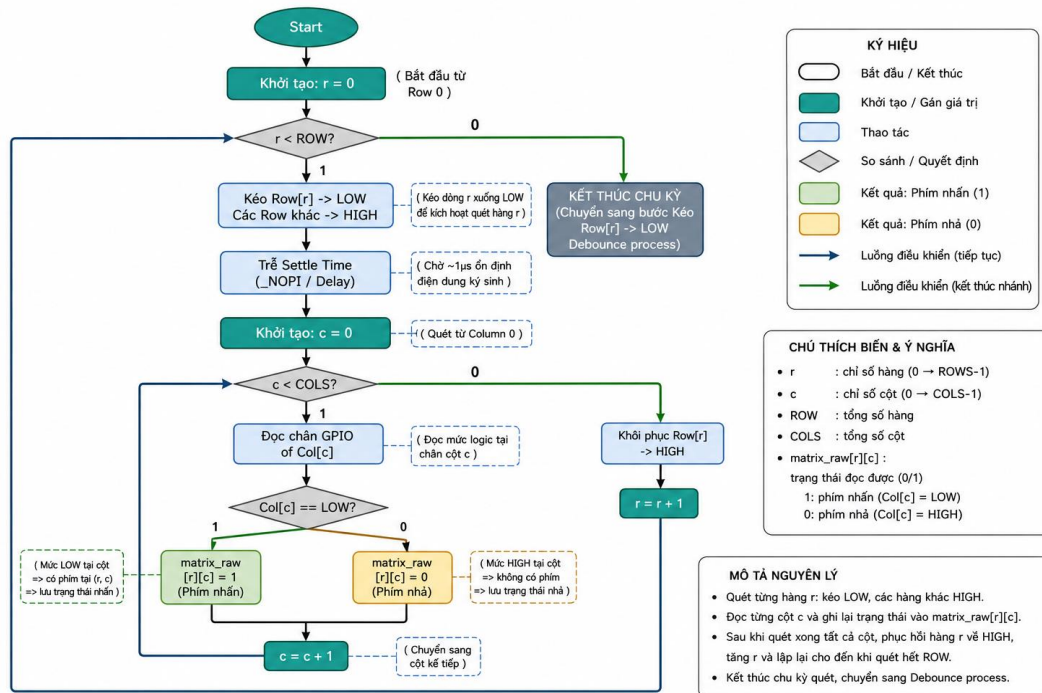
Hình 10: Layout PCB 3D của sản phẩm

5.3. Cơ chế quét và giải quyết hiện tượng Ghosting

Trong thiết kế bàn phím, **hiện tượng Ghosting** (nhấn phím này nhưng máy tính nhận nhầm thêm phím khác) là một trở ngại lớn khi người dùng nhấn tổ hợp nhiều phím.

- Linh kiện bảo vệ: Mỗi switch trong ma trận được lắp nối tiếp với một **Diode 1N4148**.
- Nguyên lý chống Ghosting:
- Diode đóng vai trò là "**van một chiều**", chỉ cho phép dòng điện đi từ Cột sang Hàng (theo logic Active LOW trong **matrix.c**).
 - Khi nhấn tổ hợp nhiều phím (ví dụ **phím ở Hàng 1 Cột 1 và Hàng 1 Cột 2**), Diode sẽ chặn dòng điện chạy ngược từ các hàng khác qua các cột, giúp MCU phân biệt chính xác từng tọa độ phím riêng biệt. Điều này cho phép bàn phím đạt chuẩn **N-Key Rollover** về mặt phần cứng.

5.4. Quy trình quét ma trận thực tế (theo Firmware)



Hình 11: Lưu đồ thuật toán của quy trình quét ma trận

Dựa trên hàm `matrix_scan()` trong file `matrix.c`, quy trình diễn ra như sau:

- Kích hoạt: MCU kéo Row 0 xuống mức LOW (0V), các Row còn lại giữ ở mức HIGH (3.3V).
- Đọc tín hiệu: MCU đọc trạng thái của 14 đường Column. Nếu phím tại tọa độ (Row 0, Col X) được nhấn, dòng điện sẽ chảy từ chân Input (đã kéo Pull-up) qua Switch và Diode về chân Output đang ở mức LOW, khiến chân Column đó chuyển sang mức 0.
- Ổn định: Một khoảng trễ cực ngắn (~1µs) được thêm vào để ổn định điện dung ký sinh trên đường mạch PCB trước khi đọc dữ liệu.
- Lặp lại: Quá trình này lặp lại liên tục cho đến Row 4, sau đó quay lại Row 0. Toàn bộ chu kỳ quét diễn ra trong vòng chưa đầy 1ms.

Nhận xét kỹ thuật: Việc kết hợp giữa Layout ANSI gọn gàng và ma trận có Diode bảo vệ giúp bàn phím của bạn vừa mang tính di động cao, vừa đạt được độ tin cậy của các dòng bàn phím cơ chuyên dụng cao cấp.

6. Thiết kế phần mềm (Firmware Design)

Hệ thống phần mềm được xây dựng trên nền tảng STM32Cube, vận hành theo kiến trúc Super Loop với chu kỳ quét cố định 1ms, đảm bảo tính thời gian thực và độ trễ đồng nhất.

6.1. Thuật toán quét và khử nhiễu (Matrix & Debounce)

6.1.1. Cơ chế quét ma trận Active LOW

```
void matrix_scan(void) {
    for (int r = 0; r < ROWS; r++) {

        // 1. KÍCH HOẠT ROW ACTIVE LOW: Kéo chân Row hiện tại xuống mức
        LOW (0V)
        HAL_GPIO_WritePin(row_ports[r], row_pins[r], GPIO_PIN_RESET);

        // 2. XỬ LÝ ỔN ĐỊNH TÍN HIỆU (SETTLING TIME): Trễ ngắn để triệt
        tiêu điện dung ký sinh
        /* ~1µs settle @ 72MHz – thay bằng DWT nếu cần chính xác hơn */
        for (volatile int d = 0; d < 72; d++);

        // 3. ĐỌC TÍN HIỆU CÁC COLUMN: Đọc 14 chân Column (đã cấu hình
        Internal Pull-up)
        for (int c = 0; c < COLS; c++) {
            // Nếu chân Column bị kéo xuống LOW (0) -> Phím được nhấn
            -> Ghi nhận giá trị là 1
            matrix_raw[r][c] = (HAL_GPIO_ReadPin(col_ports[c],
            col_pins[c]) == GPIO_PIN_RESET) ? 1 : 0;
        }

        // 4. KHÔI PHỤC TRẠNG THÁI: Kéo chân Row trả lại mức HIGH
        (3.3V) trước khi sang Row tiếp theo
        HAL_GPIO_WritePin(row_ports[r], row_pins[r], GPIO_PIN_SET);
    }
}
```

Thay vì kiểm tra toàn bộ 61 phím cùng lúc, firmware thực hiện quét tuần tự (Time-division multiplexing) trong file matrix.c:

- Quy trình: Lần lượt từng chân trong 5 chân Row (PA0-PA4) được kéo xuống mức logic 0. Khi đó, 14 chân Column (PB0-PB13) với điện trở kéo lên nội bộ (Internal Pull-up) sẽ được đọc giá trị.
- Xử lý ổn định tín hiệu: Một đoạn mã trễ ngắn (volatile int d) được chèn vào ngay sau khi đổi trạng thái Row. Điều này cực kỳ quan trọng để triệt tiêu điện dung ký sinh trên đường mạch PCB (settling time), tránh việc đọc nhầm giá trị của hàng trước đó.

6.1.2. Thuật toán Integrator Debouncing

```

void debounce_process(void) {
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {

            // Đọc trạng thái thô hiện tại từ ma trận (0 hoặc 1)
            uint8_t raw = matrix_raw[r][c];

            // HIỆU QUẢ: Nếu trạng thái thô thay đổi so với lần scan
            trước (chớm rung/nhiều)
            if (raw != raw_prev[r][c]) {
                /* Tín hiệu vừa thay đổi → reset bộ đếm về 0, chưa tin
                tưởng */
                counter[r][c] = 0;
                raw_prev[r][c] = raw; // Cập nhật trạng thái thô gần
                nhất
            }
            // ĐIỀU KIỆN XÁC LẬP: Tín hiệu ổn định (không đổi so với
            lần scan trước)
            else {
                /* Tín hiệu ổn định → tăng counter */
                if (counter[r][c] < DEBOUNCE_TIME) {
                    counter[r][c]++;

                    // Khi bộ đếm tích lũy liên tiếp đạt ngưỡng
                    DEBOUNCE_TIME (10 lần)
                    if (counter[r][c] == DEBOUNCE_TIME) {
                        /* Đủ điều kiện xác lập → chấp nhận và cập nhật
                        vào mảng debounced */
                        debounced[r][c] = raw;
                    }
                }
            }
        }
    }
}

```

Bản chất của switch cơ học là hai lá đồng va chạm, gây ra hiện tượng rung (bounce) trong khoảng 5-10ms đầu tiên. File debounce.c giải quyết vấn đề này bằng thuật toán Counter-based Integrator:

- Nguyên lý: Mỗi phím trong ma trận có một bộ đếm riêng (counter[r][c]).
- Điều kiện xác lập: Tín hiệu chỉ được coi là "đã nhấn" hoặc "đã thả" nếu trạng thái đó duy trì liên tục trong 10 lần quét (DEBOUNCE_TIME = 10).

- Hiệu quả: Nếu có một xung nhiễu ngẫu nhiên làm thay đổi trạng thái trong tích tắc, bộ đếm sẽ bị reset về 0 ngay lập tức, ngăn chặn hoàn toàn hiện tượng "double click" không mong muốn

6.2. Xử lý logic bàn phím (Keyboard Processor)

Module keyboard.c đóng vai trò chuyển đổi dữ liệu thô từ ma trận thành các gói tin có nghĩa theo chuẩn USB.

6.2.1. Quản lý Layer và Phím chức năng (Fn)

```
void keyboard_process(void) {
    hid_report_t report;
    memset(&report, 0, sizeof(report));

    uint8_t fn_held = 0;
    uint8_t key_idx = 0;
    uint8_t overflow = 0;    /* nhiều hơn 6 phím → báo ERR_OVF */

    /* =====
     * CƠ CHẾ QUÉT HAI BƯỚC (TWO-PASS SCANNING)
     * ===== */

    /* ---- BƯỚC 1 (Pass 1): Duyệt qua ma trận để tìm phím KC_FN ----
    */
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            if (!debounced[r][c]) continue; // Phím không nhấn -> Bỏ
            qua

            // Tra cứu mã phím tại Layer 0 để xác định phím Fn
            uint8_t kc = keymap_get(0, r, c); /* Fn luôn check layer 0
            */

            if (IS_FN(kc)) {
                fn_held = 1; // Ghi nhận phím Fn đang được nhấn giữ
            }
        }
    }

    // Chuyển đổi trạng thái Layer dựa trên kết quả kiểm tra Fn
    current_layer = fn_held ? 1 : 0;

    /* ---- BƯỚC 2 (Pass 2): Duyệt lần hai để lấy Keycode từ Layer hiện
    tại ---- */
}
```

```

    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            if (!debounced[r][c]) continue; // Phím không nhấn -> Bỏ
qua
            // Lấy mã Keycode tương ứng với current_layer vừa xác định
ở Bước 1
            uint8_t kc = keymap_get(current_layer, r, c);

            if (kc == KC_NONE || kc == KC_NO || IS_FN(kc)) {
                continue; // Bỏ qua các ô trống hoặc chính phím Fn
            }

            // Phân loại mã phím và đóng gói vào HID Report để gửi qua
USB
            if (IS_MODIFIER(kc)) {
                report.modifier |= modifier_bit(kc); // Đặt bitmask cho
phím hỗ trợ (Ctrl, Shift, Alt...)
            } else if (IS_NORMAL(kc)) {
                if (key_idx < MAX_KEYS_IN_REPORT) {
                    report.keys[key_idx++] = kc; // Thêm phím thường
vào danh sách 6KRO
                } else {
                    overflow = 1; // Quá 6 phím thường -> Báo lỗi tràn
                }
            }
        }
    }

    // (Đoạn sau xử lý kiểm tra trạng thái overflow và gọi hàm gửi
usb_hid_send...)
}

```

Firmware hỗ trợ kiến trúc đa tầng (Multi-layer):

- Cơ chế quét hai bước:
 - Duyệt qua ma trận để tìm phím KC_FN. Nếu phát hiện đang giữ phím Fn, biến current_layer sẽ được chuyển từ 0 sang 1.
 - Duyệt lần thứ hai để lấy Keycode từ bảng keymaps tương ứng với layer hiện tại.
- Tính linh hoạt: Điều này cho phép một phím vật lý đảm nhận nhiều chức năng (ví dụ: Phím 1 ở Layer 0 sẽ là phím F1 ở Layer 1).

6.2.2. Cơ chế 6-Key Rollover (6KRO) và Modifiers

```
typedef struct {
    uint8_t modifier;    /* Byte 0: Bitmask quản lý các phím bổ trợ
(Ctrl, Shift, Alt...) */
    uint8_t reserved;    /* Byte 1: Để trống (theo đặc tả USB HID) */
    uint8_t keys[6];     /* Byte 2 -> 7: Mảng chứa tối đa 6 mã phím
thường (6KRO) */
} hid_report_t;
```

Để tối ưu hóa gói tin gửi đi, firmware phân loại mã phím thành hai nhóm

- **Modifier Keys:** Gồm Ctrl, Shift, Alt, Gui. Các phím này được quản lý dưới dạng Bitmask trong byte đầu tiên của báo cáo USB. Điều này cho phép nhấn giữ tất cả các phím Modifier cùng lúc.

```
static uint8_t modifier_bit(uint8_t kc) {
    switch (kc) {
        /* Ánh xạ các phím bổ trợ bên trái (Left Modifiers) */
        case KC_LCTL: return MOD_LCTRL; // Tương đương (1 << 0) -
Bit 0
        case KC_LSFT: return MOD_LSHIFT; // Tương đương (1 << 1) -
Bit 1
        case KC_LALT_K: return MOD_LALT; // Tương đương (1 << 2) -
Bit 2
        case KC_LGUI_K: return MOD_LGUI; // Tương đương (1 << 3) -
Bit 3

        /* Ánh xạ các phím bổ trợ bên phải (Right Modifiers) */
        case KC_RCTL: return MOD_RCTRL; // Tương đương (1 << 4) -
Bit 4
        case KC_RSFT: return MOD_RSHIFT; // Tương đương (1 << 5) -
Bit 5
        case KC_RALT_K: return MOD_RALT; // Tương đương (1 << 6) -
Bit 6
        case KC_RGUI_K: return MOD_RGUI; // Tương đương (1 << 7) -
Bit 7

        default: return MOD_NONE; // Không phải phím bổ trợ
(0x00)
    }
}
```

- Normal Keys: Tối đa 6 phím thường (A, B, C...) sẽ được xếp vào mảng 6 byte tiếp theo. Nếu vượt quá 6 phím, hệ thống sẽ gửi mã 0x01 (Error Roll Over) để báo cho máy tính biết giới hạn phần cứng đã đạt tới.

```

/*
=====
==
* TRÍCH ĐOẠN ĐÓNG GỐI BÁO CÁO HID VÀ XỬ LÝ LỖI TRÀN PHÍM (OVERFLOW)
*
=====
== */
uint8_t key_idx = 0; /* Biến đếm chỉ số phím thường hiện tại trong
mảng keys[6] */
uint8_t overflow = 0; /* Cờ hiệu báo trạng thái tràn giới hạn phím
thường */

// ... (Vòng lặp duyệt qua ma trận phím đã lọc nhiễu debounced) ...
uint8_t kc = keymap_get(current_layer, r, c);

if (IS_MODIFIER(kc)) {
    /* Phím bổ trợ: Tích lũy bitmask vào byte đầu tiên */
    report.modifier |= modifier_bit(kc);
}
else if (IS_NORMAL(kc)) {
    /* Phím thường: Kiểm tra điều kiện giới hạn 6KRO */
    if (key_idx < MAX_KEYS_IN_REPORT) {
        /* Còn chỗ trống: Điền mã phím vào mảng và tăng chỉ số đếm
*/
        report.keys[key_idx++] = kc;
    } else {
        /* Vượt quá 6 phím thường: Kích hoạt cờ báo tràn phần cứng
*/
        overflow = 1;
    }
}

// ... (Kết thúc vòng lặp quét ma trận) ...

/* XỬ LÝ KHI PHÁT HIỆN LỖI TRÀN PHÍM (Nhấn từ 7 phím thường trở lên) */
if (overflow) {
    /* Đè dữ liệu toàn bộ mảng keys bằng mã lỗi chuẩn quốc tế 0x01
(KC_ERR_OVF) */
    for (int i = 0; i < MAX_KEYS_IN_REPORT; i++) {
        report.keys[i] = KC_ERR_OVF;
    }
}

```

```
/* Chuyển giao gói tin HID Report hoàn chỉnh sang driver USB để truyền về PC */  
usb_hid_send(&report);
```

6.3. Giao tiếp USB HID và Tối ưu hóa băng thông

Giao tiếp được thực hiện thông qua module usb_hid.c, tuân thủ nghiêm ngặt mô hình Producer-Consumer:

- Cấu trúc gói tin (HID Report):
 - Byte 0: Modifier keys (8 bits).
 - Byte 1: Reserved (luôn là 0).
 - Byte 2-7: Mảng chứa 6 Keycodes đang được nhấn.
- Cơ chế gửi dựa trên thay đổi (Differential Reporting):
 - Để tối ưu hóa băng thông đường truyền, firmware triển khai cơ chế Differential Reporting (Báo cáo vi sai). Hệ thống sẽ duy trì một biến tĩnh bộ đếm mang tên last_report để lưu trữ trạng thái của gói tin ở chu kỳ gửi thành công ngay trước đó.

Vùng khai báo biến static và hàm khởi tạo bộ đếm được thiết lập ở đầu tệp tin usb_hid.c như sau:

```
/* Vị trí khai báo: Phần đầu file usb_hid.c */  
#include "usb_hid.h"  
#include "usbd_hid.h"  
#include "usb_device.h"  
#include <string.h>  
  
/* Biến điều khiển cấu trúc USB Endpoint Full Speed do STM32CubeMX sinh ra */  
extern USBD_HandleTypeDef hUsbDeviceFS;  
  
/*  
=====
```

```
==  
* CƠ CHẾ CACHE REPORT ĐỂ TỐI ƯU HÓA BĂNG THÔNG (DIFFERENTIAL  
REPORTING)  
*  
=====
```

```
== */  
static hid_report_t last_report; /* Bộ đếm lưu trữ trạng thái gói tin của lần gửi trước đó */  
static uint8_t initialized = 0; /* Cờ hiệu ghi nhận trạng thái khởi tạo hệ thống USB */
```



```

/**
 * @brief Khởi tạo module USB HID và xóa trống bộ đệm lưu trữ
 */
void usb_hid_init(void) {
    /* Xóa sạch vùng nhớ đệm last_report về 0x00 khi khởi động hệ thống
    */
    memset(&last_report, 0, sizeof(last_report));
    initialized = 1;
}

```

- Hàm thư viện chuẩn memcmp() được sử dụng để so sánh trực tiếp toàn bộ vùng nhớ 8 byte của báo cáo mới phát sinh với báo cáo cũ đang lưu trong cache. Lệnh gọi driver gửi dữ liệu qua hàm phần cứng USB_D_HID_SendReport chỉ được phép thực thi khi và chỉ khi phát hiện có sự khác biệt (người dùng nhấn thêm phím mới hoặc nhả phím cũ).

Thuật toán gửi dữ liệu có điều kiện này được hiện thực chi tiết như sau:

```

/* Vị trí hiện thực: Thân hàm usb_hid_send() trong file usb_hid.c */
/**
 * @brief Gửi gói tin HID Report về máy tính dựa trên sự thay đổi trạng
    thái phím
 * @param report: Con trỏ chứa cấu trúc gói tin ma trận phím hiện tại
    (8 byte)
 */
void usb_hid_send(const hid_report_t *report) {
    /* Kiểm tra an toàn: Nếu module chưa sẵn sàng thì thoát hàm */
    if (!initialized) return;

    /* CƠ CHẾ GỬI DỰA TRÊN THAY ĐỔI:
    * So sánh từng byte dữ liệu giữa report mới và last_report (bộ đệm
    cũ).
    * Nếu kết quả trả về bằng 0 (hai gói tin giống hệt nhau - trạng
    thái tĩnh),
    * firmware lập tức thoát hàm (return) để chặn truyền tải dư thừa
    lên bus USB. */
    if (memcmp(report, &last_report, sizeof(hid_report_t)) == 0)
        return;

    /* TRUYỀN DỮ LIỆU QUA CỔNG PHẦN CỨNG:
    * Lệnh gọi Driver HAL chỉ thực thi khi bước memcmp phía trên phát
    hiện sự khác biệt. */
    USB_D_HID_SendReport(
        &hUsbDeviceFS,
        (uint8_t *)report,
        sizeof(hid_report_t)
    );
}

```

```

);

/* CẬP NHẬT TRẠNG THÁI CACHE:
 * Sao chép gói tin vừa truyền vào last_report để làm mốc đối chiếu
cho chu kỳ kế tiếp. */
memcpy(&last_report, report, sizeof(hid_report_t));
}

/**
 * @brief Gửi gói tin trống (Empty Report) để báo hiệu người dùng đã
nhả toàn bộ phím
 */
void usb_hid_send_empty(void) {
    hid_report_t empty = {0}; // Tạo gói tin 8 byte chứa toàn bộ giá
trị 0x00
    usb_hid_send(&empty);
}

```

- Lợi ích:
 - Giảm thiểu xung đột trên bus USB và tiết kiệm tài nguyên CPU cho cả vi điều khiển và máy tính, đặc biệt quan trọng khi hoạt động ở Polling rate cao (1000Hz)

7. Đánh giá kết quả dự án

Bảng 3: Đánh giá kết quả thực hiện các công việc trong dự án

STT	Nội dung công việc	Kết quả thực tế đạt được
0	Tìm hiểu về specs về các sản phẩm bàn phím thương mại và tìm các bài báo, github	Đạt 100%
1	Thiết kế phần cứng	- Hoàn thành sơ đồ nguyên lý (Schematic) ma trận 5x14. (đạt 100%) - Đã lựa chọn và có đầy đủ linh kiện cốt lõi (đạt 100%) (STM32F103C8T6, 1N4148). - Layout PCB 100% ANSI đã hoàn thiện bản vẽ (đạt 100%). - Thiết kế và xây dựng vỏ bao ngoài của sản phẩm (đạt 100%).
2	Phát triển Firmware	- Cấu hình thành công USB HID trên STM32CubeMX. (đạt 100%) - Mã nguồn quét ma trận đã chạy mô phỏng và kiểm tra được tín hiệu trên các chân GPIO (PA/PB). (đạt 100%) - Thuật toán Debounce cơ bản đã được viết xong (đạt 100%).
3	Logic & Hệ thống	- Thiết lập xong cấu trúc mảng Keymap cho Layer 0. (đạt 100%) - Xác định được phương pháp quản lý Modifier Keys (Ctrl, Alt, Shift) trong Report Descriptor của USB. (đạt 100%)

7.1. Kết quả hoàn thiện của sản phẩm



Hình 11: Sản phẩm khi đã hoàn thiện

Bảng 4: Bảng thống kê phần trăm sử dụng AI trong các chương của báo cáo

Chương	Phần trăm sử dụng AI
1	25%
2	20%
3	20%
4	10%
5	10%
6	20%
7	0%